NEA - Group Creator for School Projects

Amelie Machin

Contents

1 Analysis **1**
1.1 Problem statement 1
1.2 Research 2
1.2.1 Sources of inspiration 2
1.2.2 Algorithm 4
1.2.3 Inputting data 5
1.2.4 Accounts 5
1.2.5 Internal data storage 5
1.2.6 External data storage 5
1.2.7 GUI library 6
1.3 Dialogue 7
1.4 Objectives 8
1.5 Modelling 9

2 Design **13**
2.1 GUI 13
2.2 Databases 20
2.3 Algorithm 20
2.3.1 Pseudocode 20
2.3.2 Diagrams 22

1 Analysis

1.1 Problem statement

To form the perfect grouping for a group project within a school, many factors need to be taken into consideration to prevent students from feeling incapable of completing the project. These factors include, but are not limited to, skills of the students, skills required by the project, and number of students in the group.

This project aims to create a tool to aid teachers in ensuring the students are separated into groups where the students are best able to get along and combine their individual skills to achieve their project's aims. The user can input the data that they want to be taken into consideration by the program and it will

output a set of groups. This is in the form of a table on a GUI - to present the results in a readable way - and an option to export the data.

This program will not find, assume, or generate information about the students or project, it will only work with the data inputted to assign a group of students to a project.

Students will also benefit from this tool as they would be able to apply their skills to a project in a group of people they can enjoy time with, rather than being stuck on a project they have no interest in with people they do not get along with.

1.2 Research

1.2.1 Sources of inspiration

1. Random group generator from ClassTools.net [Fig. 1]
   https://www.classtools.net/random-group-generator/

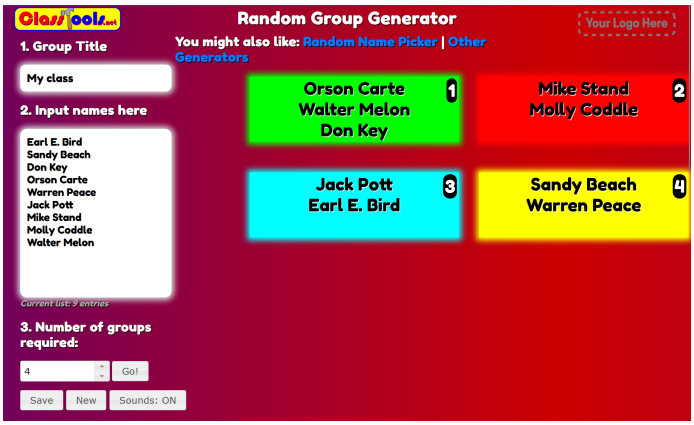


Figure 1: ClassTools's random group generator tool with inputs and outputs visible on the same page

- This tool is used frequently by my teachers to create random groups for group activities.
- The GUI is very simple to use as there is little data that needs to be entered, so can all fit down the left of the screen.
- The output is very easy to read as each group is visually separated using both coloured boxes and numbering. [Obj. 3b]
- However, this tool is too simple because it lacks any way to group with conditions, which is part of my scope, as they are purely randomised. This means that teams could quit often be unfairly balanced in terms of skill, or group students together that are known to not work well together. The teacher's only option to solve this is

to re-roll until the output is suitable enough for the task manually. This gave me the idea for the algorithm to be the simulation of this re-rolling and selection of optimal outputs.

2. keamk [Fig. 2] [Fig. 3]
   https://www.keamk.com/random-team-generator

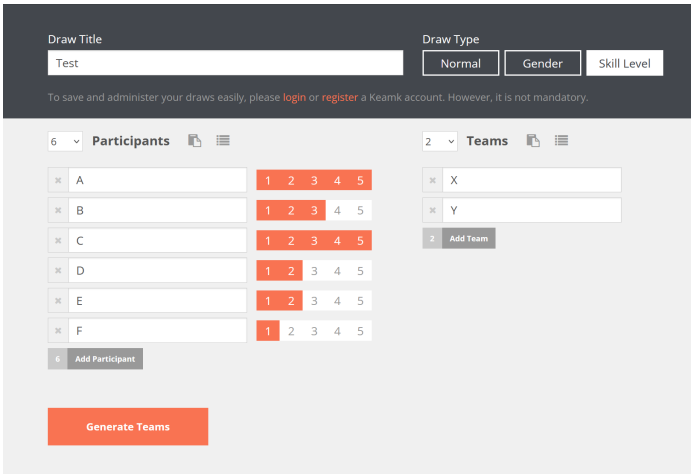


Figure 2: Keamk's menu for inputting data



Figure 3: Keamk's outputs

- This tool creates groups that all have a balanced level of skill.
- I like how the skill level is inputted as a rating between 1-5 as this is intuitive for the user.
- I also like the extra details that are outputted, such as how many are in each group next to the group name and the skill levels next to each individual. [Obj. 3a]
- Furthermore, the option to go back to make small changes to the data inputted and get new groupings was very quick and useful. [Obj. 1g]
- However, this tool can only create balanced groups, not groups with different objectives as is in my scope, and there is no way to assign a certain number of participants to each group, as the program puts

the same number (or one away) of participants in each group. [Obj. 2c-d]

These tools influenced me to reduce the complexity of my GUI, and gave me ideas such as to keep groups visually separate using colour as well as tables, adding the option to go back and edit the inputted data later, and some summary statistics to include with the outputs.

1.2.2 Algorithm

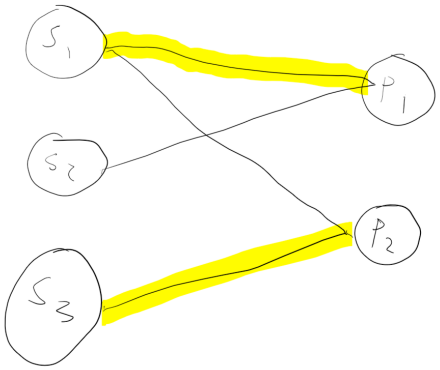


Figure 4: A bipartite graph with 3 student nodes on the left, 2 project nodes and the right, and some edges between them. A matching algorithm has been applied, with the matching highlighted in yellow. This has not met the scope because a student has not been matched.

I have researched into representing students and projects into bipartite graphs, and then using matching algorithms such as the alternate path algorithm to join students to projects [Fig. 4]. However, this will only match one student to one project, not groups so is not suitable. I then researched applying the algorithm repeatedly to graphs with randomised students, but this means that some students will never be joined into a group purely based on chance and that the groups formed are such that each individual is best for the project, not that the overall group is best suited for the project, so teamwork will not as good as it could be. I then researched using a grouping algorithm, such as https://help.feedbackfruits.com/hc/en-us/articles/23527084211346-Group-Formation-Algorithm-Explainer to create similar groups then using bipartite matching algorithms to match them. However this only creates groups with similar skill sets, not skill sets that vary based on the project.

I decided on a genetic algorithm where each individual is made up of a possible solution and the fitness score is a calculation between how suited each group is to the assigned project. The solutions with lower fitness scores are discarded and the higher ones crossover to form new solutions. Some of these

solutions then have random mutations, which involve swapping of students between projects. New fitness scores are calculated and the process repeats a predetermined number of times [Fig. 8]. I chose this because genetic algorithms are very adaptable to varying conditions and constraints, such as the ones that the user chooses in this program. This will allow the program to test different combinations of projects and students, test different group sizes for a project, and swap in and out optional projects to find better and better solutions until it reaches a high-quality solution that fits all the user's needs. This fits the project scope far better than all previously researched algorithms.

1.2.3 Inputting data

For the inputting of data, I have decided that it can be inputted through txt and csv files [Obj. 1a], manually on the GUI [Obj. 1b], or any combination of those methods [Obj. 1c]. This is because the data for different individuals and different projects could be stored in different ways within the same system, and manual tweaks may be needed by the user to ensure all data is as is needed by the application. I will also include the option to import the data from an Excel spreadsheet at the request of my third party [Obj. 1b].

1.2.4 Accounts

I have decided against implementing an account system because I don't think it will provide any value to the program. Accounts are typically used for ensuring data can only be accessed by the users with the correct permissions. As this program will be designed to be run locally, all the data will be stored and only accessible on the machine the program was run on, so the OS can handle user security.

1.2.5 Internal data storage

I chose to store the data about the students and projects in their own classes, that are aggregated within an 'individual' class. The student and project classes contain their names, skills and the project will have the recommended group size. The individual will contain a project object, student objects, and current fitness score as well as methods for calculating fitness scores, crossover with other individuals, and mutations. [Fig. 6]

This keeps the attributes and methods stored together in the program to make the code easier to read and understand by a programmer.

1.2.6 External data storage

[Obj. 4] I have researched into whether information about projects and students should be stored somewhere, and if so, how. I have concluded that they should be stored in relational databases that are accessible and modifiable by entering the details into the graphical interface, which are passed as parameters into

SQL statements. These can then be loaded as a list of checkboxes that can be selected and deselected by the user for that run. [Obj. 1c]

I chose to store the data because being able to make minor changes to the data and then re-running the program quickly is a very valuable feature that could save users' time and energy as they would not have to go through the entire process of entering every piece of data just for small changes to some of the data. This feature is present in keamk's grouping tool, and I found it very useful in my testing.

I chose databases to store the data because I have more experience working with relational databases and SQL then any other form of data storage, such as in files. The data is structured so databases would allow for efficient access and modification of specific entries.

1.2.7 GUI library

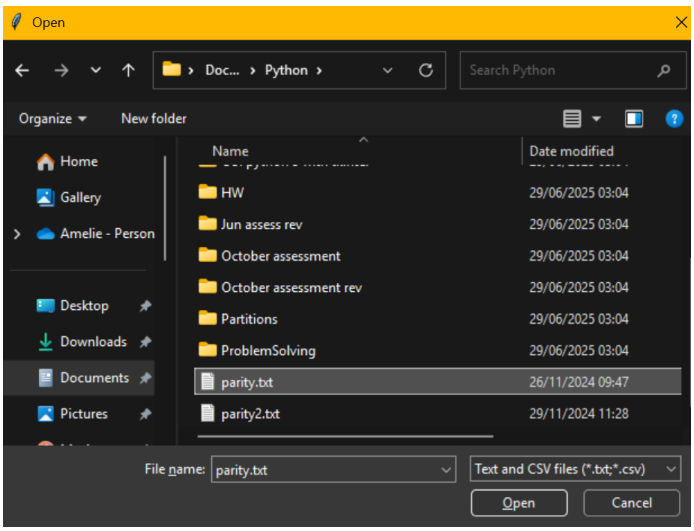


Figure 5: Tkinter's file dialog, configured to only show .txt and .csv files

Between the two main python GUI libraries, pygame and Tkinter, I chose Tkinter because of the larger library of classes of pre-made, customisable widgets that can be used more easily to produce a simple, easy-to-use tool, such as a dialog that allows users to graphically navigate through their files to find their data file [Fig.5].

On the other hand, with my prior experience with pygame, I have found it to be more suited towards building games and graphically-complex tools, which is not what I am aiming for in this project. I am aiming to have a GUI that is quick and simple to create and edit, and has user-friendly features for inputting, editing, and reading data. I also have more experience with Tkinter,

which will reduce the amount of time I spend on the GUI and allow me to focus on developing and improving the grouping algorithm and user experience.

1.3 Dialogue

My third party is Dr Ed Horncastle, the lead teacher for the Exeter Mathematics Certificate (EMC) at my school, which is where groups of 3 or 4 students work together on research projects set by University academics or companies. He is suitable because he has to make these groups based on the skills of the students and match them to projects every year, so he has a thorough understanding of how this tool could work and would benefit from using this tool. I interviewed him and asked him the following questions:

1. What is the current process for creating EMC groups?
    - All the metrics to consider are gathered into a spreadsheet and then groups are formed by hand.
    - Some projects don't end up getting groups assigned, especially those that have been set by a company that already has another project with an assigned group.
2. What are these metrics?
    - Students' own choice
    - There must be a mix of genders
    - The projects must be relevant to the A-level studies of that student. Someone who does not take computer science shouldn't be given a computer science based project
    - Those that work hard and those that laze off more easily should be mixed together. If this is their first project, this is inferred from GCSE results, otherwise it is based off the results of previous projects.
    - Special educational needs
    - Timetabling restrictions
    - Students do not get put into groups with others they have worked on projects with before
3. How are the student choices measured?
    - For some of the projects, the students give their top 3 choices of projects. Starting with the least popular project, the groups are formed by trying to minimise the sum of the positions the student's ranked the project in.
4. How is the data about the students and projects currently stored?
    - Excel spreadsheet

5. How would you prefer to input the data?
    - Student choices from a Microsoft Form and all other data from a Excel spreadsheet
6. How much data and in what format do you think the data should be outputted?
    - In tables with the project name, student names and summaries of information about each group such as gender ratio, how many take computer science or physics, and how many have special education needs.
    - As a simplified register in alphabetical order
7. How useful would storing project/student details locally, that can be loaded when needed, be for you?
    - Very
8. Are there any other features that would be useful for this tool to include?
    - Some extra features that could be added is timetabling for what order the projects are presented in, and itinerary for group trips out of school.

After this, I decided to include Excel spreadsheets as an input method [Obj. 1b], include options for projects to not have groups assigned despite being a part of the algorithm [Obj. 1f], take genders into account [Obj. 2e], have summary statistics presented alongside the groups [Obj. 3a], have the option to view the groups without any extra information [Obj. 3b], store the data locally for re-use at another time [Obj. 4].

I decided not to include student choice, student personality, special education needs, timetabling restrictions, or preventing grouping the same students together across multiple as adding too many restrictions to the algorithm will make it take it longer to run to create a set of groups that best matches all of these restrictions. These are the restrictions that were lower on Dr Horncastle's priorities. I will also not include timetabling or scheduling as these deviate quite far from my project's scope.

1.4 Objectives

1. Data can be entered into the program from external sources
    (a) Data can be entered manually into the GUI
    (b) Data can be entered through txt, csv, and Excel spreadsheet files
    (c) Data can be loaded from stores from previous uses of the program
    (d) Data can be entered through a combination of other methods
    (e) Student data must contain a name, gender, and skills

i. Name must only contain alphanumeric characters and certain whitelisted characters

(f) Project data must contain a name, maximum group size, minimum group size, whether or not the project must have a group or not, and skill requirements

i. Name must only contain alphanumeric characters and certain whitelisted characters

ii. Maximum group size must be greater than 0

iii. Minimum group size must be greater than or equal to 0

(g) Data can be added, edited, and removed at any time during the program's runtime

(h) Data must be entered before the algorithm is run

2. Students must be grouped together and to a project

(a) All students must be put into a group

(b) All students must have at least one skill that will allow them to contribute to the project, when possible

(c) Projects must not have more students assigned to it than the maximum limit inputted by the user

(d) Projects must not have less students assigned to it than the minimum limit inputted by the user

(e) Groups must have a mix of genders

3. The resulting groups can be viewed

(a) This can be through coloured tables displayed on the GUI that contain the project name, student names, number of students, gender ratio, and skill range

(b) This can be through a register that displays projects in alphabetical order, grouped with its students in alphabetical order, all coloured to separate from each other, and no other information.

(c) Both above options must automatically be saved as ... files in the local file directory.

4. All data about projects and students must be stored

1.5 Modelling

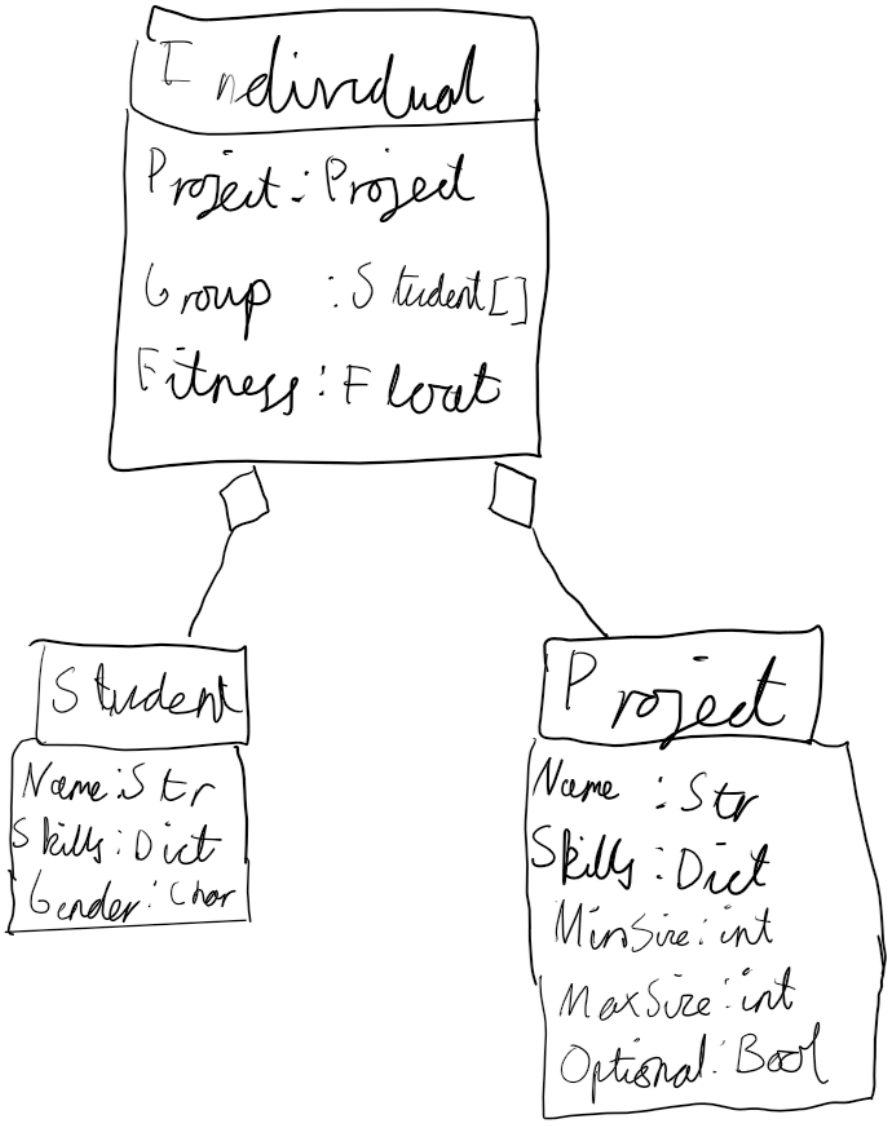


Figure 6: Class diagram showing relationships between the classes and their attributes used by the genetic algorithm.

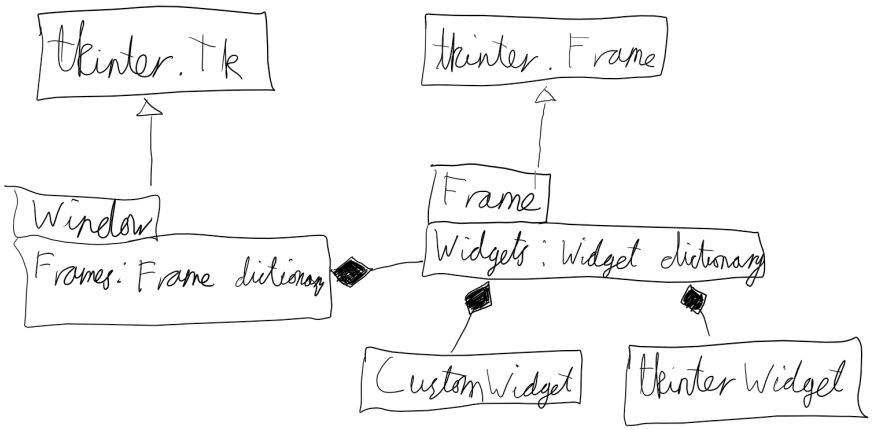


Figure 7: Class diagram showing relationships between the classes that form the GUI

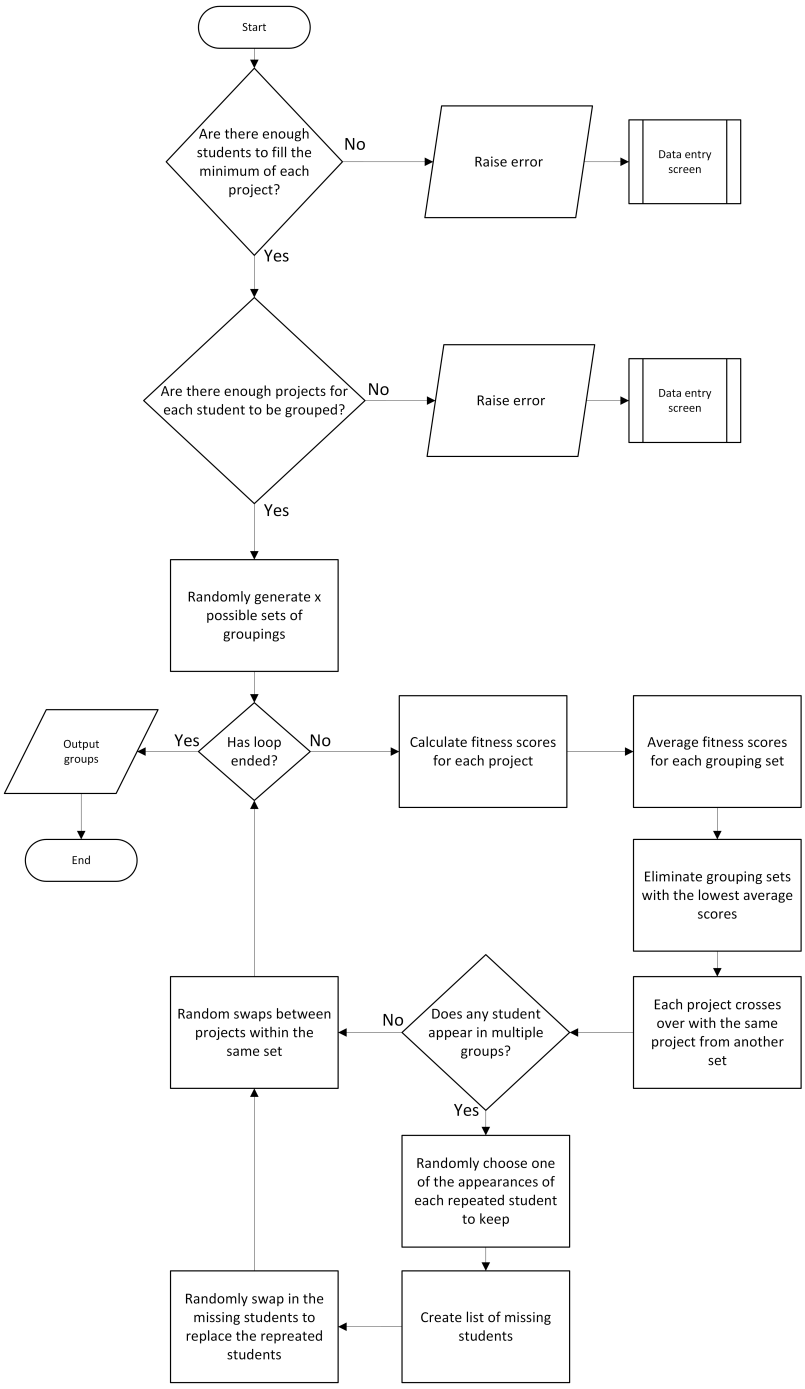


Figure 8: Flowchart showing the process for the genetic algorithm

2 Design

2.1 GUI

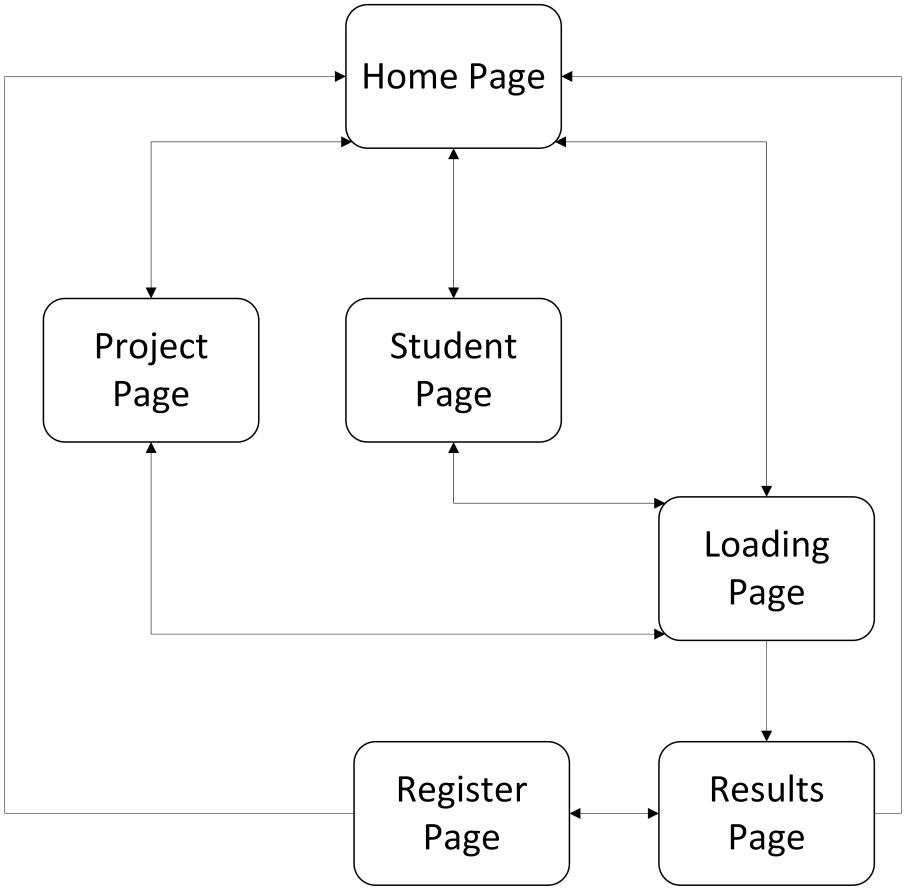


Figure 9: Page navigation diagram showing which pages can navigate to each other. Note: The student and project pages can only navigate to the previous page.

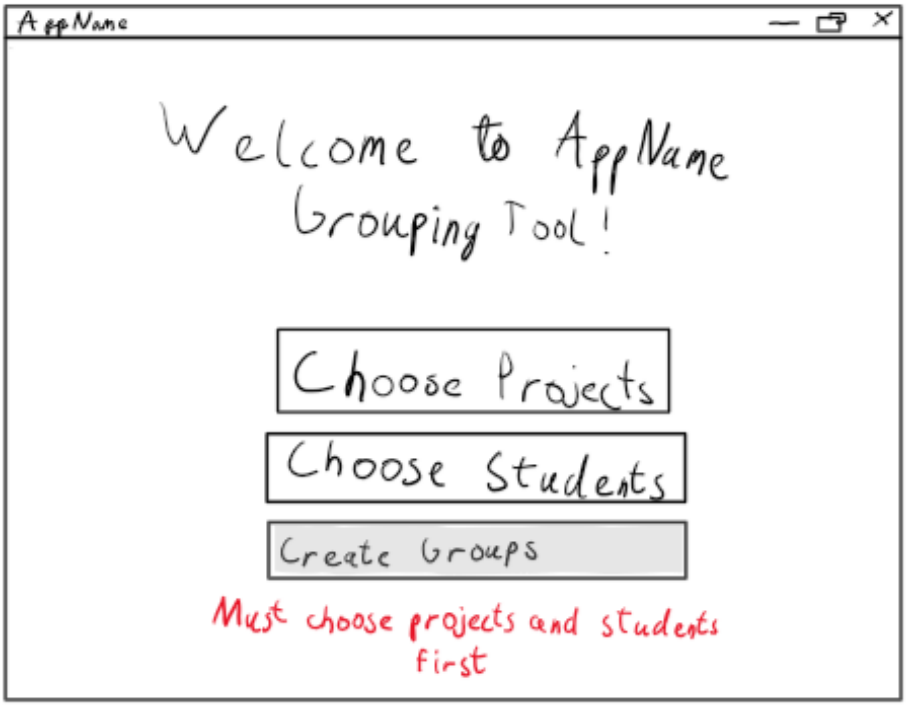


Figure 10: Sketch of the homepage layout that is the first page seen upon opening the app. The "Choose Projects" and "Choose Students" lead to the projects and students pages, respectively. The "Create Groups" button leads to the loading page and runs the grouping algorithm with the students and projects the user has selected. It is greyed out and not clickable unless at least one project and student has been selected [Obj. 1h].

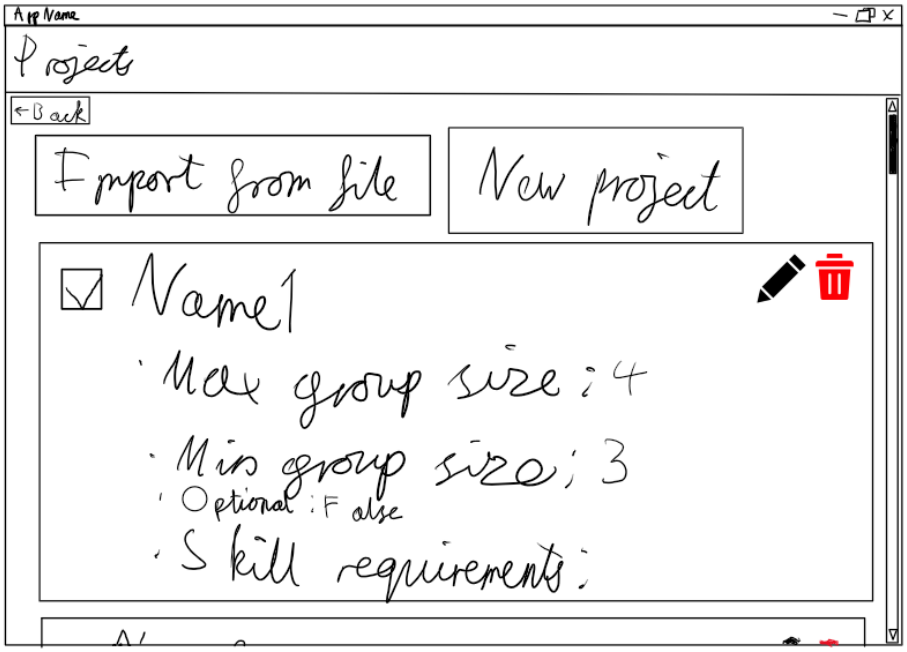


Figure 11: Sketch of the project page layout. The projects in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import project data from supported files [Obj. 1b] and to manually enter project details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The project details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each project in the list features a checkbox, allowing users to select which of the projects in the database will be used in the grouping algorithm. Projects not selected will not have groups matched to them. This page also features a back button to go back to the home page.

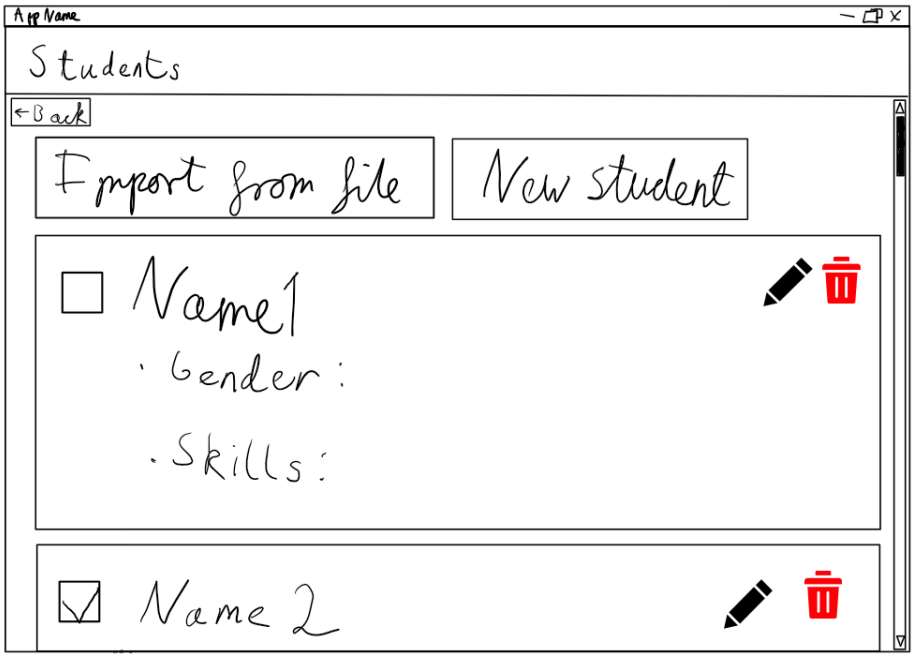


Figure 12: Sketch of the student page layout. The students in the scrollable list are all those that are currently stored in the database [Obj. 1c]. There are options to import student data from supported files [Obj. 1b] and to manually enter student details at the top [Obj. 1a], which then get added to the database [Obj. 4] and the visual list [Obj. 1d]. The student details can also be edited or deleted from the database by pressing the buttons [Obj. 1g], which run parametrised SQL. Each student in the list features a checkbox, allowing users to select which of the students in the database will be used in the grouping algorithm. Students not selected will not be put into groups. This page also features a back button to go back to the home page.

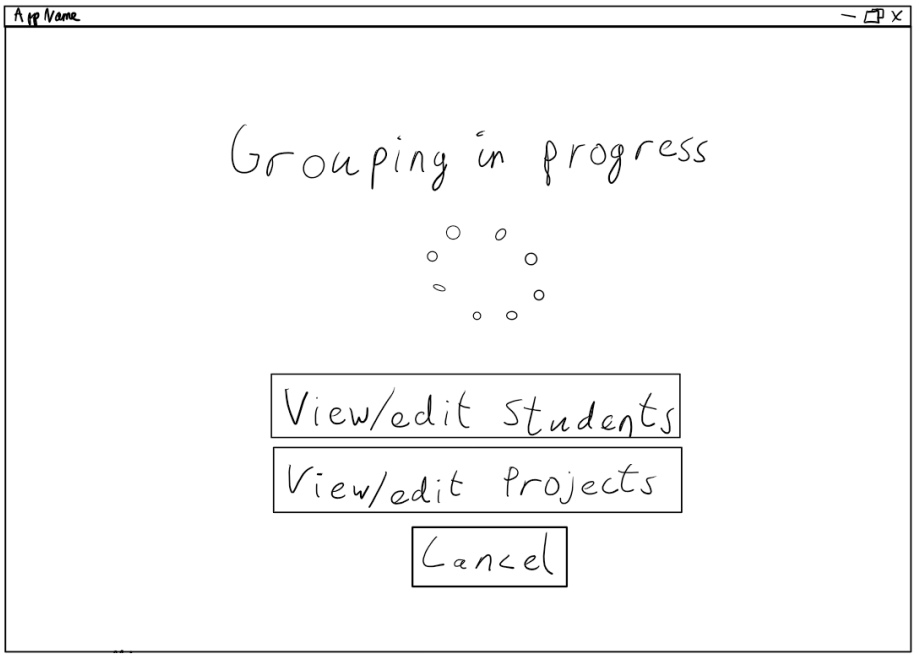


Figure 13: Sketch of the loading page layout. This page features a loading animation, and three buttons. The "View/edit Students" and "View/edit Projects" buttons lead to the Students and Projects pages, respectively [Obj. 1g]. The "Cancel" button ends the algorithm and leads to the home page. Once the algorithm has finished successfully, it moves onto the results page.

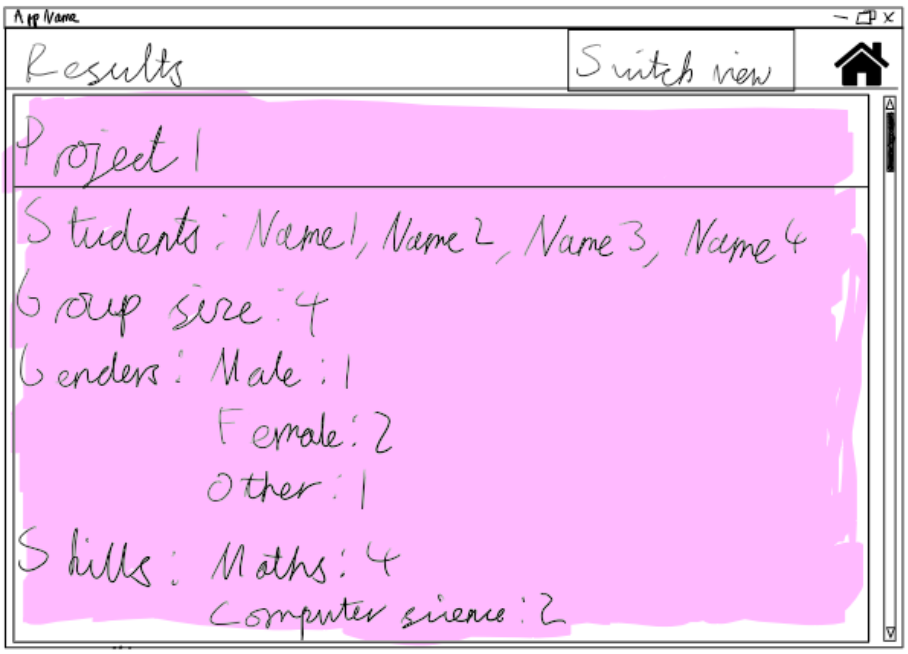


Figure 14: Sketch of the results page layout. It features a scrollable list of all the projects with the students grouped to it and summary statistics about the group, such as group size, gender ratio, and skills [Obj. 3a]. The "Switch view" leads to the register page and the home button leads to the home page.

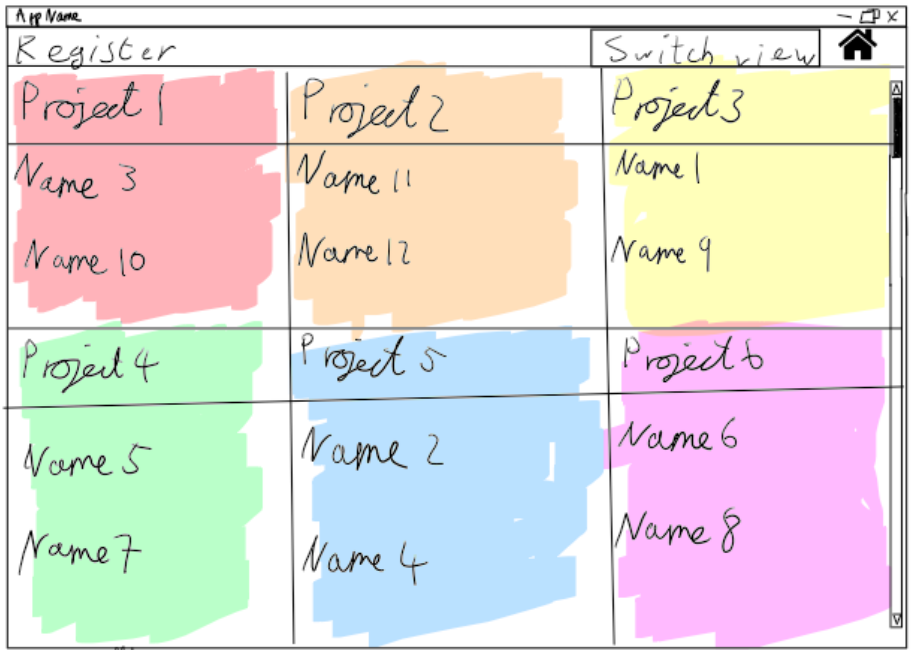


Figure 15: Sketch of the register page layout.

2.2 Databases

- Students(StudentID, FirstName, LastName, Gender)
- StudentSkills(StudentID, Skill)
- Projects(ProjectID, Name, MaxSize, MinSize, Optional)
- ProjectSkills(ProjectID, Skill)

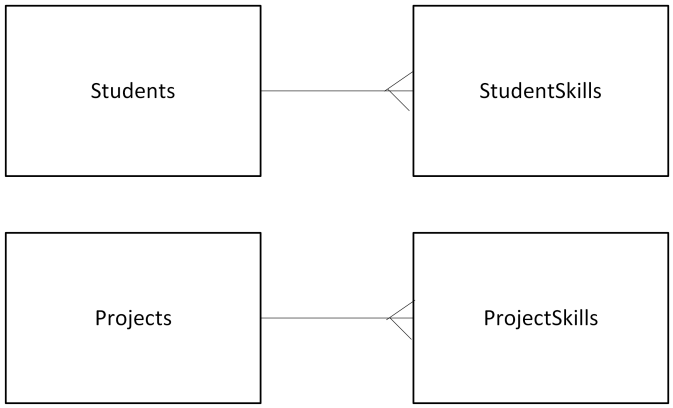


Figure 16: An entity relationship diagram showing the one-to-many relationships between the Students and Projects entities and the store of their skills to ensure all fields are atomic so the database is normalised.

2.3 Algorithm

2.3.1 Pseudocode

Pseudocode for the function that runs when on the loading page. It ensures it is possible to find groupings with the given requirements, then runs a genetic algorithm to find a good set of groupings.

```
TARGET_FITNESS = integer
MAX_ITERATIONS = integer
POPULATION_SIZE = integer


students = set of 'student' objects
projects = set of 'project' objects


#Ensure there aren't too many or too few students to meet
    ↪ requirements
```

```
minStudents = 0
maxStudents = 0

REPEAT for all projects:
    maxStudents += project.maxSize
    IF (not project.optional):
        minStudents += project.minSize

#Check for too few students
IF (length of students < minStudents):
    DisplayError("Error: Only {length of students} to
        ↪ fill a minimum of {minStudents} places.")
    HomePage()

#Check for too many students
ELSE IF (length of students > maxStudents): #Checks for
    ↪ too many
    DisplayError("{length of students} is too many to fit
        ↪  in {maxStudents} places.")
    HomePage()


#Generate initial grouping sets and average fitness
    ↪ scores
population = list
REPEAT for i = 0 to POPULATION_SIZE - 1:
    groups = GenInitialGroups(projects, students)
    avgFit = (SUM fitness of each group)/(length of
        ↪ groups)
    APPEND groups,avgFit to population

SORT population by avgFit

#Genetic algorithm
REPEAT for i = 1 to MAX_ITERATIONS:
    #Ends early when one set of groupings is good enough
    IF (highest avgFit > TARGET_FITNESS):
        BREAK

    #Eliminate the worst half of grouping sets
    population = first (POPULATION_SIZE FLOORDIV 2) of
        ↪ population
    new_pop = []

    #Create next population
    REPEAT for j = 1 to 90% of POPULATION_SIZE:
```

```
        #Crossover random groupings
        groups = Mate(random group from population,
            ↪ random group from population)

        #Ensure all students appear once
        valid = verifyGroupings(groups)
        IF (not valid):
            fixGroupings(groups)

        #Random mutations to prevent convergence towards
            ↪ a local best solution
        Mutate(groups)

        #Find the average fitness score for each new
            ↪ grouping
        avgFit = (SUM fitness of each group)/(length of
            ↪ groups)
        APPEND groups,avgFit to new_pop

    #Combine the crossover with 10% elitism
    population = new_pop + top 10% of population
    SORT population by avgFit


return group with highest avgFit
```

2.3.2 Diagrams

Figure 17: NOT FINISHED A class diagram showing the attributes and methods of the classes used by the genetic algorithm, and the relationships between each class.